

Sécurité Avancée

Chiffrement homomorphe

David Papa

Chiffrement classique vs homomorphe

- Classique :
 - nécessite un déchiffrement préalable pour permettre un traitement ou une analyse → faille de sécurité potentielle
- Homomorphe :
 - effectue des calculs directement sur des données chiffrées, sans jamais devoir les décrypter
 - Résultat identique que si on avait déchiffré puis effectué l'opération

Utile en cryptographie

- Algorithme d'Euclide étendu :
 - calcule le plus grand commun diviseur
 - permet de trouver les coefficients de Bézout, deux entiers x et y tels que :
 - $\text{pgcd}(a, b) = xa + by$
 - Avec x l'inverse modulaire de $a \bmod b$
 - $ax \equiv 1 \bmod b$
- Exponentiation rapide :
 - Permet de calculer $a^b \bmod n$ pour des grands nombre
 - Simplifie en divisant l'exposant b :
 - $13 = 1101_2 = 8 + 4 + 0 + 1 \Rightarrow a^{13} = a^8 a^4 a^1$
 - On peut calculer les carrés successifs de $a \bmod n$ pour trouver rapidement n'importe quelle valeur $a^b \bmod n$

Utile en cryptographie

- Test de primalité de Miller-Rabin :
 - Algorithme probabiliste qui permet de vérifier rapidement si un nombre est probablement premier → pratique pour des grands nombres
 - petit théorème de Fermat :
 - Si n est premier et $1 < a < n - 1$:
 - $a^{n-1} \equiv 1 \pmod{n}$
 - Avec ce principe, si $a^{n-1} \not\equiv 1 \pmod{n}$, n est forcément pas premier
 - On répète l'opération pour réduire l'aléatoire

RSA

- Génération des clés :
 - On choisit deux grands nombres premiers, notés p et q
 - On calcule $n = p \cdot q$
 - On calcule $\lambda(n)$ qui correspond au plus petit multiple commun entre $p - 1$ et $q - 1$
 - On choisit un entier e , premier avec $\phi(n) = (p - 1) \cdot (q - 1)$ et inférieur à celui-ci ($e < \phi(n)$)
 - On calcul d , l'inverse modulaire de e modulo $\phi(n) \rightarrow$ algorithme d'Euclide étendu
- On obtient la clé publique (e, n) et la clé privée (d, n)

RSA

- Chiffrement :
 - Pour un message m :
 - $c = m^e \bmod n$
- Déchiffrement :
 - $m = c^d \bmod n$
 - Car $c^d \equiv m^{ed} \bmod n$, avec $ed = 1 + k(p - 1)(q - 1)$
 - $m^{ed} \equiv m^{1+k(p-1)(q-1)} \equiv m * m^{k(p-1)(q-1)} \equiv m \bmod p \equiv m \bmod q$
 - L'entier $m^{ed} - m$ est donc un multiple de p et de q , qui sont premiers distincts, donc de leur produit $pq = n$

RSA homomorphique

- Multiplication :
 - $\varepsilon(m) = m^e \bmod n$
 - $\varepsilon(m_1) * \varepsilon(m_2) = m_1^e * m_2^e \bmod n = (m_1 m_2)^e \bmod n = \varepsilon(m_1 m_2)$

ElGamal

- Génération des clés :
 - On choisit un grand nombre premier p , on définit q tel que $q = p - 1$ et g un générateur du groupe multiplicatif $\mathbb{Z}_p^*$
 - On choisit une clé privée x au hasard entre $1 \leq x \leq q - 1$
 - On calcule $h = g^x \bmod p$
- On obtient notre clé publique (p, q, g, h) et notre clé privée x

ElGamal

- Chiffrement :

- Pour un message chiffré M , on doit d'abord l'adapter pour qu'il fasse partie du groupe cyclique $\mathbb{Z}_p^*$ (simplifier dans notre code par un modulo, pas reversible)
- On génère un nombre y aléatoire compris entre $1 \leq y \leq q - 1$
- On calcule :
 - $c_1 = g^y \bmod p$
 - $c_2 = m * h^y \bmod p$

- Déchiffrement :

- Avec $s = c_1^x = g^{xy} = h^y$
- $m = c_2 * s^{-1} \bmod p$
- Car $c_2 * s^{-1} = m * h^y * c_1^{q-x} = m * g^{xy} * g^{y(q-x)} = m * g^{xy} * g^{qy} * g^{-xy} = m * g^{qy} \bmod p \equiv m \bmod p$

ElGamal homomorphique

- Multiplication :
 - $\varepsilon(m) = (g^y, m * h^y)$
 - $\varepsilon(m_1) * \varepsilon(m_2) = (g^{y_1}, m_1 * h^{y_1}) * (g^{y_2}, m_2 * h^{y_2}) = (g^{y_1+y_2}, (m_1 m_2) * h^{y_1+y_2}) = \varepsilon(m_1 m_2)$

Goldwasser-Micali

- Système de chiffrement par bit
- Génération des clés :
 - On choisit deux grands nombres premiers p et q
 - On calcule $N = p \cdot q$
 - On cherche un x qui satisfait la fonction du symbole de Legendre :
 - $\frac{x}{p} = \frac{x}{q} = -1$
 - On peut facilement calculer cela car :
 - $a^{\frac{p-1}{2}} \equiv \frac{a}{p} \text{ mod } p$
- On obtient la clé publique (x, N) et la clé privée (p, q)

Goldwasser-Micali

- Chiffrement :
 - Avant de pouvoir chiffrer notre message m , il faut le transformer en série de bits (i correspond au $i^{\text{ème}}$ bit)
 - On génère un y_i entre $2 \leq y_i \leq N - 1$
 - On calcule le bit encrypté :
 - $c_i = y_i^2 * x^{m_i} \bmod N$
 - On obtient une série bits encryptés
- Déchiffrement :
 - on teste si c_i est un résidu quadratique modulo N :
 - $m_i = \begin{cases} 0 & \text{pour } c_i^{\frac{p-1}{2}} \bmod p = c_i^{\frac{q-1}{2}} \bmod q = 1 \\ 1 & \text{sinon} \end{cases}$
 - On convertie notre série de bits en nombre pour retrouver m

Goldwasser-Micali homomorphique

- XOR :
 - $\varepsilon(b) = y^2 x^b \bmod N$
 - $\varepsilon(b_1) * \varepsilon(b_2) = y_1^2 x^{b_1} * y_2^2 x^{b_2} \bmod N = (y_1 y_2)^2 x^{b_1 + b_2} \bmod N = \varepsilon(b_1 \oplus b_2)$
 - $b_1 + b_2 \rightarrow$ addition binaire

Benaloh

- Goldwasser-Micali groupé
- Génération des clés :
 - Définir un $r \geq 2$ qui indique le nombre de bits groupés
 - On choisit deux grands nombres premiers p et q tel que :
 - $(p - 1) \bmod r = 0, \gcd\left(r, \frac{p-1}{r}\right) = 1, \gcd(r, q - 1) = 1$
 - On calcule $n = p \cdot q$ et $\phi = (p - 1) \cdot (q - 1)$
 - On choisit un $y \in \mathbb{Z}_n^*$ avec $y^{\frac{\phi}{r}} \not\equiv 1 \bmod n$ (simplifiable avec les facteurs de r)
 - On définit $x = y^{\frac{\phi}{r}} \bmod n$
- On obtient note clé publique (y, n) et notre clé privée (ϕ, x)

Benaloh

- Chiffrement :
 - On définit un $u \in \mathbb{Z}_n^*$
 - $c = y^m u^r \bmod n$
- Déchiffrement :
 - On calcule :
 - $a = c^{\frac{\phi}{r}} \bmod n$
 - On cherche un $m = \log_x(a) \Rightarrow x^m \equiv a \bmod n$

Benaloh homomorphique

- Addition :
 - $\varepsilon(m) = y^m u^r \bmod n$
 - $\varepsilon(m_1) * \varepsilon(m_2) = (y^{m_1} u_1^r) * (y^{m_2} u_2^r) \bmod n = y^{m_1+m_2} (u_1 u_2)^r \bmod n = \varepsilon(m_1 + m_2 \bmod r)$
 - Dans les groupes cycliques, toutes les puissances de y sont cycliques modulo r

Paillier

- Génération des clés :
 - On choisit deux grands nombres premiers p et q tel que :
 - $\gcd(pq, (p-1)(q-1)) = 1$
 - On calcule $n = p \cdot q$ et $\lambda = \text{lcm}(p - 1, q - 1)$ où lcm indique le plus petit multiple commun
 - On choisit un nombre aléatoire $g \in \mathbb{Z}_{n^2}^*$
 - On calcule $\mu = \left(L(g^\lambda \bmod n^2) \right)^{-1} \bmod n$ avec $L(x) = \frac{x-1}{n} \rightarrow$ inverse avec algorithme d'Euclide étendu
- On obtient la clé publique (n, g) et la clé privée (λ, μ)

Paillier

- Chiffrement :
 - Il faut que $m < n$
 - On génère un $r < n$ avec $\gcd(r, n) = 1$
 - $c = g^m * r^n \bmod n^2$
- Déchiffrement :
 - On utilise la même fonction $L(x) = \frac{x-1}{n}$
 - $m = L(c^\lambda \bmod n^2) * \mu \bmod n$

Paillier homomorphique

- Addition :

- $\varepsilon(m) = g^m r^n \bmod n^2$
- $\varepsilon(m_1) * \varepsilon(m_2) = (g^{m_1} r_1^n) * (g^{m_2} r_2^n) \bmod n^2 = g^{m_1+m_2} (r_1 r_2)^n \bmod n^2 = \varepsilon(m_1 + m_2)$

- Multiplication :

- $\varepsilon(m) = g^m r^n \bmod n^2$
- $\varepsilon(m_1)^{m_2} \bmod n^2 = \varepsilon(m_2)^{m_1} \bmod n^2 = \varepsilon(m_1 m_2) \bmod n^2$
- L'exposant n'est pas encrypté

Librairie TenSEAL

- Bibliothèque Python pour effectuer des calculs sur des données chiffrées
- Interface simple pour le chiffrement, les opérations homomorphes, et le déchiffrement
- Il faut initialiser un contexte de chiffrement (paramètres de chiffrement)
 - Type BFV adapté aux entiers

```
# Initialisation du contexte de chiffrement BFV
context = ts.context(
    ts.SCHEME_TYPE.BFV,
    poly_modulus_degree=8192,
    plain_modulus=786433,
    coeff_mod_bit_sizes=[60, 40, 40, 60]
)
context.generate_galois_keys()
context.generate_relin_keys()
```

Librairie TenSEAL

- Nombres chiffrés sous forme de liste pour permettre d'optimiser les opérations

```
# Chiffrement  
encrypted_num1 = ts.bfv_vector(context, [num1])  
encrypted_num2 = ts.bfv_vector(context, [num2])
```

- Opérations identiques que avec des valeurs claires (soustraction aussi possible)

```
encrypted_sum = encrypted_num1 + encrypted_num2 # Addition homomorphe  
encrypted_prod = encrypted_num1 * encrypted_num2 # Multiplication homomorphe
```

- Décryptage avec la librairie :

```
# Déchiffrement  
sum_decrypted = encrypted_sum.decrypt()  
prod_decrypted = encrypted_prod.decrypt()
```

Résultats

- Temps d'exécution :

Cryptosystème	Addition	Multiplication	XOR	Unité
Pas d'encryption	0.0	0.0	0.0	ms
RSA	-	1.0014	-	ms
ElGamal	-	0.9997	-	ms
Goldwasser-Micali	-	-	228.996	ms
Benaloh	0.9987	-	-	ms
Paillier	3.9999	4.0002	-	ms
TenSEAL	0.4508	29.8412	-	ms
Moyenne encryptée	1.8165	8.9606	228.996	ms

TABLE 1 – Tableau des temps d'exécution par cryptosystème.

Résultats

- Espace mémoire occupé :

Cryptosystème	Taille du nombre	Après opération	Unité
Pas d'encryption	28	28	octets
RSA	92	92	octets
ElGamal	56	56	octets
Goldwasser-Micali	2208	2208	octets
Benaloh	92	160	octets
Paillier	160	160	octets
TenSEAL	334405	-	octets

TABLE 2 – Tableau de l'espace mémoire occupé par les nombres chiffrés par cryptosystème.

- Benaloh pas réduite au modulo n et peut donc occuper plus d'espace après opération

Conclusion

- RSA et ElGamal : relativement simple et efficace pour des opérations de multiplications
- Goldwasser–Micali : permet des calculs sur chaque bits indépendamment mais ses performances chute
- Benaloh : très efficace pour des additions homomorphes
- Paillier : moins performant que nos autres cryptosystèmes mais plus polyvalent
- TenSEAL : possibilité de calculs homomorphes complets, optimisé pour effectuer plusieurs opérations en même temps, très efficace pour les additions mais attention à la mémoire